

Statistical Learning Methods



Module 1.2: Prediction for SLM

AUTHOR

John Mayberry

Introduction

Last class, we discussed the concept of a statistical model and introduced our first model in R: a simple linear model

$$Y = \beta_0 + \beta_1 X + \varepsilon$$

where ε is a normal random variable with standard deviation σ . We shall henceforth call this model a simple linear model (SLM).

We introduced the function `lm` to estimate coefficients in an SLM from a given set of observations and used it to start exploring the relationship between *trump.vote* and *non.white* in the *election2016* dataset:

```
library(tidyverse)
df_url="https://raw.githubusercontent.com/jmayberr/math133-stat-learning-fall2026/refs/heads/main,
election2016=read.csv(df_url)
y=election2016$trump.vote
x=election2016$non.white
lm(y~x)
```

Call:

```
lm(formula = y ~ x)
```

Coefficients:

(Intercept)	x
0.5811	-0.2655

The output tells us that the best linear equation for predicting $y = \text{trump.vote}$ from $x = \text{non.white}$ is $\hat{y} = 0.5811 - 0.2655x$. For example, if I have a state that is 50% white, then the predicted trump vote (as a proportion) would be

```
0.5811-0.2655*0.5
```

```
[1] 0.44835
```

The real test of a model then is how well it predicts new data, but that often first involves assessing how it fits the data used to estimate model parameters in the first place (called the training data). Before we get into that, a couple comments on R.

The most efficient way to use `lm` and other modeling functions is to pass the formula and data directly as inputs:

```
lm(trump.vote ~ non.white, data=election2016)
```

Call:

```
lm(formula = trump.vote ~ non.white, data = election2016)
```

Coefficients:

(Intercept)	non.white
0.5811	-0.2655

However, if you want use the model to predict new values it is helpful to first save the output of `lm` to a variable and then refer to that variable in future calls:

```
fit_lm=lm(trump.vote ~ non.white, data=election2016)
print(fit_lm)
```

Call:

```
lm(formula = trump.vote ~ non.white, data = election2016)
```

Coefficients:

(Intercept)	non.white
0.5811	-0.2655

One advantage of this approach is that instead of manually calculating the predicted values $\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x$ for new or old observations, we can use the `predict` function. For example, to predict the Trump vote when the non-white proportion is 0.5, we do

```
new_df=data.frame(non.white=0.5)
predict(fit_lm, new_df)
```

```
1
0.4484042
```

That may seem more tedious than just using the formula, but it won't when the formulas get more complicated. The `predict` function is also helpful because we can pass multiple x values and get all the predictions at once:

```
new_df=data.frame(non.white=seq(0,1,0.1))
predict(fit_lm, new_df)
```

1	2	3	4	5	6	7	8
0.5811319	0.5545863	0.5280408	0.5014953	0.4749497	0.4484042	0.4218587	0.3953132
9	10	11					
0.3687676	0.3422221	0.3156766					

Or if you want to determine the predicted values for all points in the original dataset, leave the second input blank and this is done by default (first six shown below):

```
yhat_lm=predict(fit_lm)
head(yhat_lm)
```

1	2	3	4	5	6
0.4882225	0.4696406	0.4510588	0.5121135	0.4192041	0.4988407

This will be useful in the next section!

Prediction Metrics on Training Set

The data we use to fit the model is called the training set. There are numerous metrics we could introduce to measure how well our model fits the training set. We introduced one in the first notes, namely

$$SSE = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

For scaling purposes, it is more common to use the Mean Squared Error (MSE) defined as the average squared error per observation:

$$MSE = SSE/n$$

This can be calculated using the predicted outputs as follows

```
y = election2016$trump.vote
yhat_lm = predict(fit_lm)
mse = mean((y-yhat_lm)^2)
print(mse)
```

```
[1] 0.008186537
```

Unfortunately, the MSE on its own has the meaningless units of $(y \text{ units})^2$ (variance anyone?) so it is more common to look at its square root, the R(oot)MSE:

```
rmse <- sqrt(mse)
print(rmse)
```

```
[1] 0.09047949
```

This says that our predictions are off by about 0.09 per state on average, although not exactly, since squaring the errors before taking the square root is not the same as simply averaging the absolute errors which instead leads to the Mean Absolute Deviation (best acronym in statistics: MAD):

```
mad=mean(abs(y - yhat_lm))
print(mad)
```

```
[1] 0.07823563
```

The difference comes down to how each metric penalizes errors. RMSE squares the errors before averaging, which gives extra weight to large mistakes, while MAD treats all errors equally regardless of size. This is similar to the difference between the ℓ_2 and ℓ_1 norms if you know about such things (if not, they will appear later in the course). Both are valid ways of measuring model accuracy; RMSE has more theoretical appeal but MAD is much easier to explain to your friends.

The final, and perhaps most commonly used metric for simple linear models, is R^2 (sometimes called the Coefficient of Determination but usually just R-squared). We'll first give the calculation and then the explanation:

```
ybar=mean(y)
sst=sum((y-ybar)^2)
sse=sum((y-yhat_lm)^2)
r2=1-sse/sst
print(r2)
```

```
[1] 0.178141
```

R^2 gives the *proportion of variance* explained by the model. To explain this phrase, SST (Sum of Squares Total) represents the overall variance in y values. It is the inherent variability in outcomes and the goal is to explain as much of that variance as possible with our models. SSE is the overall error in our model predictions or the variance left after we account for the relationship with our predictor. SSE/SST is therefore the fraction of the overall variance in outcomes that is NOT explained by the model and $1 - \text{SSE/SST}$ is the fraction that IS explained by our model. A perfect model would have an SSE of 0 and hence, an R^2 value of 1. Here is a function you can use to compute the R^2 for a model given an outcome y and predicted values $\hat{y}$:

```
get_r2=function(y,yhat){
  ybar=mean(y)
  sst=sum((y-ybar)^2) # total variance in y
  sse=sum((y-yhat)^2) # variance unexplained by model
  r2=1-sse/sst      # proportion of variance explained by model
  r2
}

#Example usage from scratch
y=election2016$trump.vote
fit_lm=lm(trump.vote~non.white,data=election2016)
```

```
yhat=predict(fit_lm)
r2=get_r2(y,yhat)
print(r2)
```

```
[1] 0.178141
```

We'll need this in the next section. You can also get the R^2 value for the training data using the summary function:

```
sum_lm=summary(fit_lm)
print(sum_lm$r.squared)
```

```
[1] 0.178141
```

Note that R^2 is just another way of looking at SSE/MSE/RMSE. Any one of these four is sufficient when comparing models. The advantage of R^2 is that it is dimensionless so it could be interpreted more universally. Here are some general guidelines for deciding what is “large” (generated with the help of AI, but complemented with my experience in statistical consulting):

Guidelines for Interpreting R^2

- **Large** ($R^2 \geq 0.7$): The model explains a large proportion of variance. Common in fields like physics or engineering where relationships tend to be more deterministic (e.g., natural sciences¹ often see $R^2 > 0.8$).
- **Moderate** ($0.3 \leq R^2 < 0.7$): The model explains a moderate proportion of variance. Frequently seen in fields like economics or other social sciences, where real-world data involve many unobserved factors.
- **Small** ($R^2 < 0.3$): The model explains a small proportion of variance, but can still be meaningful. In fields like psychology, sociology, or behavioral sciences, an R^2 of 0.1–0.2 may actually be considered quite good since human behavior is complex and influenced by many factors.

Training-Testing Validation

Testing set validation means to split off some fraction of your data (usually 20-30%) for testing how well the model predicts “new data”. The observations used for testing are called the *hold-out set* or sometimes just *testing* data. Splitting the data into training and testing sets is called *partitioning* the data. In R, this can be done by deciding on the fraction p to use for your training set and then sampling $n * p$ random rows from the dataset as the training set, using the excluded rows for testing:

```
set.seed(45)
n=nrow(election2016)
p=0.7 # Decide on fraction to keep for training, in this case 70%
train_rows=sample(n,0.7*n) # Choose training rows
```

```
train_df=election2016[train_rows,] # Extract training data
test_df= election2016[-train_rows,] # Extract testing data
```

This would be called a “70-30 split” because we used 70% for training and 30% for testing. We fit our model to the training set

```
fit_lm_train=lm(trump.vote~non.white,data=train_df)
```

And calculate any validation metrics like R^2 on the testing set

```
y_test=test_df$trump.vote
yhat_test=predict(fit_lm_train,test_df)
r2_test=get_r2(y_test,yhat_test)
print(r2_test)
```

```
[1] -0.2826803
```

Validation metrics will generally be smaller on the test set BUT less likely to be misleadingly large due to overfitting (more on this topic to come). We can see in this example that R^2 came out negative on the testing set which means that our model did terrible at predicting new data (it would have been better to just guess the mean of y for all our observations).

Every time we do the training-testing set split we will get a different value for R^2 on the testing set due to the randomness of our partition. For example, running the same code again yields

```
n=nrow(election2016)
p=0.7
train_rows=sample(n,0.7*n)
train_df=election2016[train_rows,]
test_df= election2016[-train_rows,]
fit_lm_train=lm(trump.vote~non.white,data=train_df)
y_test=test_df$trump.vote
yhat_test=predict(fit_lm_train,test_df)
r2_test=get_r2(y_test,yhat_test)
print(r2_test)
```

```
[1] -0.04216912
```

Well, the answer was still negative. Overall, this repetition seems to be a sign that despite passing the visual test and (as we’ll see in the next section) the inference test, *non.white* is not a very good predictor of *trump.vote*. Later we’ll discuss cross-validation, a method for repeatedly performing training-testing splits in a systematic way. For now, get some practice trying creating these splits.

Practice

Import the [Advertising.csv](#) file into R (Source: [Introduction to Statistical Learning with R](#)). This simulated dataset contains sales (in terms of thousands of units sold) for a number of different stores in a chain as well as money spent (in thousands of dollars) on advertising in TV, radio, and newspaper (clearly an outdated marketing scheme).

1. Fit separate SLM for the three different predictors of sales (TV, radio, and newspaper). Calculate R^2 for each of the three and compare.
 2. Split the data into training and testing sets using an 80-20 split. Refit models for TV and radio to the training data and compute R^2 on the testing data for both.
 3. Create a scatterplot of sales vs x where x was the better of the two predictors in Step 2.
-

Footnotes

1. In some areas of biology, such as ecology, smaller R^2 values of 0.3–0.7 can also be meaningful) [↩](#)